SoupsSnacks v2 - Full-Stack Reference & Learning Guide

Document version: 1.0
Project: Soups, Snacks & More - Order Management System
Repository: SoupsSnacks_v2
Generated for: Python engineers growing into Django + React full-stack development

Table of Contents

1. Executive Summary

SoupsSnacks v2 is a working web application for managing a home-based food business. It was built with Django 5.2 (backend API) and React 19 (single-page frontend), communicating over REST + JSON with Django session cookies for authentication.

As a core Python developer, you already understand models, business logic, and data. This app adds:

- HTTP APIs (Django REST Framework)
- Browser UI (React components, routing, state)
- Cross-origin security (CORS, CSRF)
- Role-based access (admin, operator, cook)

The goal of this document is to be your offline reference when enhancing the app and when practicing full-stack skills until you can build similar systems independently.

2. Business Domain & Features

What the business does

Operators run a daily food service (soups, snacks, sweets, lunch, etc.) primarily for residential apartment customers in Bangalore-style communities (apartment name + block filters).

Core workflows

Workflow	Description
Catalog	Define products with selling price and ingredient cost breakdown; margins compute...
Daily menu	Pick products available on a given date; export text for WhatsApp
Orders	Create orders with line items; track status from draft -> delivered -> completed
Payments	Record cash/UPI/etc.; order payment status updates automatically
Reports	Sales, profitability, unpaid orders, inactive customers, loyalty analytics
Import	Bulk CSV/Excel for customers, products, orders, payments
Google Sheets	Sync new rows from a form-linked sheet into orders

User roles

Role	Typical user	Primary tasks
admin	Owner	Users, imports, Google sync, everything
operator	Front desk	Customers, orders, payments, reports, daily offerings
cook	Kitchen	Product catalog and cost components (menu/pricing)

3. Architecture Overview

High-level diagram

Architectural choices (why they matter)

Choice	Rationale	Trade-off
Separate frontend	React gives rich UI without Django templates	Two codebases to deploy
Session auth	Simple, secure for same-site SPA	Harder for mobile/native apps (would use ...
SQLite	Zero-config for local dev	Not ideal for high concurrent write load
DRF ViewSets	Less boilerplate for CRUD	Learning curve vs plain Django views
Domain apps	Each business area is isolated	More files, but clearer boundaries

Request lifecycle (example: create order)

- User fills form in OrderForm.js
- React calls POST /api/orders/orders/ via api.js
- Browser sends session cookie + X-CSRFToken
- Django middleware: CORS -> Session -> CSRF -> Auth
- DRF checks IsOperator permission
- OrderCreateUpdateSerializer validates nested items

- Models saved; JSON response returned
- React navigates to order detail or refreshes list

4. Technology Stack

Backend (pinned in requirements.txt)

Package	Version	Purpose
Django	5.2.12	Web framework, ORM, admin, sessions
djangorestframework	3.17.0	REST API layer
django-cors-headers	4.9.0	Allow React origin with credentials
python-dotenv	1.2.2	Load `.env` for secrets/settings

Backend (optional, used in code)

Package	Purpose
openpyxl	Read `.xlsx` in import preview
google-auth, google-api-python-client	Google Sheets sync

Frontend (package.json)

Package	Purpose
react, react-dom ^19	UI components
react-router-dom ^7	Client-side routing (`/orders`, `/catalog`, ...)
axios ^1.13	HTTP client to Django API
react-scripts 5	Create React App build tooling

What is NOT used

- No GraphQL, Redux, TanStack Query, Tailwind, Material UI
- No Docker/Kubernetes in repo (you can add later)
- No Celery/Redis (all sync processing)

5. Repository Layout

```
SoupsSnacks_v2/
??? soupssnacks/              # Django project settings & root URLs
?   ??? settings.py
?   ??? urls.py
??? accounts/                 # Custom User, login, permissions
??? customers/
??? catalog/                  # Products + cost components
??? offerings/                # Daily menus
??? orders/
??? payments/
??? reports/                  # Analytics APIViews (no models)
??? imports/                  # Bulk CSV/Excel
??? integrations/             # Google Sheets
??? frontend/src/             # React application
?   ??? App.js                # Route definitions
?   ??? contexts/AuthContext.js
?   ??? services/api.js       # Axios instance
?   ??? components/           # Layout, ProtectedRoute, etc.
?   ??? pages/                # One file pair per screen (.js + .css)
??? import_templates/         # Sample CSV files
??? tests/test_core.py
```

```
??? manage.py
??? seed_demo_data.py         # Rich demo dataset
??? requirements.txt
??? .env.example
??? docs/                     # This reference document
```

Note: README mentions a backend/ folder; in this repo Django apps live at the root beside frontend/.

6. Django Backend Deep Dive

6.1 Django concepts mapped to this project

Django concept	Where you see it	Python analogy
Project	`soupssnacks/`	Top-level package
App	`orders/`, `customers/`	Feature module
Model	`orders/models.py`	Class ? DB table
Migration	`orders/migrations/`	Schema version control
ViewSet	`orders/views.py`	Class exposing CRUD endpoints
Serializer	`orders/serializers.py`	Validation + JSON ? model
URL routing	`*/urls.py`	URL -> handler map
Middleware	`settings.py` MIDDLEWARE	Request pipeline hooks

6.2 Installed apps (settings.py)

Django built-ins: admin, auth, contenttypes, sessions, messages, staticfiles.

Third-party: rest_framework, corsheaders.

Project apps: accounts, customers, catalog, offerings, orders, payments, reports, imports, integrations.

6.3 REST Framework defaults

```
DEFAULT_PERMISSION_CLASSES = [IsAuthenticated]
DEFAULT_AUTHENTICATION_CLASSES = [SessionAuthentication]
```

Every API endpoint requires login unless explicitly overridden (e.g. login, health).

6.4 Typical ViewSet pattern

Each resource app follows:

- Model - database fields and @property for computed values
- Serializer - ModelSerializer with read-only computed fields; separate create/update serializer for nested children
- ViewSet - ModelViewSet + @action for custom endpoints (stats, toggle_active)
- urls.py - DefaultRouter registers basename -> /api/<app>/<resource>/

Example custom actions on orders:

- POST .../change_status/ - workflow transitions
- POST .../change_payment_status/ - manual override
- GET .../today/, .../pending/ - filtered lists
- POST .../import_csv/ - operator CSV import

6.5 Reports app (different pattern)

reports/ has no models. It uses class-based APIView classes that run Django ORM aggregations (Sum, Count, annotate) and return JSON or CSV HttpResponse.

This is ideal for read-only analytics without polluting domain models.

6.6 Key business logic locations

Logic	File
Order totals / profit	`orders/models.py` - properties on `Order`, `OrderItem`
Payment status sync	`payments/models.py` - `save()` / `delete()` on `Payment`
Product margins	`catalog/models.py` - `unit_cost`, `margin_percent`
Google row -> order	`integrations/google_sheets_service.py`
CSV import validation	`imports/views.py`

6.7 Management commands

Command	Usage
`python manage.py migrate`	Apply DB schema
`python manage.py createsuperuser`	Django admin user
`python manage.py create_test_users`	admin/operator/cook test accounts
`python manage.py create_sample_products`	Sample catalog
`python manage.py create_sample_customers`	Sample customers
`python seed_demo_data.py`	Full demo (--reset to wipe)

7. Database Schema & Relationships

Entity relationship (core)

```
User

Customer ??< Order ??< OrderItem >?? Product
             ?
             ???< Payment
             ???< GoogleSheetOrderRef

Product ??< ProductCostComponent
Product ??< DailyOfferingItem >?? DailyOffering

GoogleSheetConfig ??< GoogleSheetSyncLog
                ???< GoogleSheetOrderRef >?? Order

ImportLog ??> User
```

Model summary

Model	Key fields	Notes
User	username, role (admin/operator/cook)	Extends AbstractUser; table `auth_user`
Customer	name, mobile, apartment_name, block	Indexed for search/filters
Product	name, unit, category, selling_price, image_url	unique_together (name, unit)
ProductCostComponent	FK product, quantity, cost_per_unit	Rolls up to unit_cost
DailyOffering	offering_date (unique), notes	One menu per day
DailyOfferingItem	FK offering, FK product, available_quantity	
Order	order_number, customer, status, payment_status	Auto order number
OrderItem	quantity, unit_price, unit_cost_snapshot	Snapshots preserve history
Payment	order, amount, method, payment_date	Updates order payment_status
ImportLog	import_type, status, errors (JSON)	Audit trail
GoogleSheetConfig	sheet_id, field_mapping (JSON)	Column letter -> field
GoogleSheetOrderRef	config, sheet_row, order	Dedup sync

Important constraints

- PROTECT on Order.customer and OrderItem.product - prevents accidental deletes
- CASCADE on payments and order items when order deleted

- Payment validation prevents total paid > order revenue (with 0.01 tolerance)

Database file

- Engine: SQLite
- Path: db.sqlite3 at project root
- Inspect: python manage.py dbshell or DB browser tools

8. REST API Reference

Base URL: http://localhost:8000/api/

Root routes (soupssnacks/urls.py)

Prefix	App
`/api/health/`	Public health check
`/api/accounts/`	Auth & users
`/api/customers/`	Customers
`/api/catalog/`	Products, cost components
`/api/offerings/`	Daily offerings
`/api/orders/`	Orders
`/api/payments/`	Payments
`/api/reports/`	Dashboard & reports
`/api/imports/`	Bulk import
`/api/integrations/`	Google Sheets

Reports endpoints (reports/urls.py)

Path	Purpose
`reports/dashboard/`	KPIs for home dashboard
`reports/sales/`	Sales over date range
`reports/customers/`	Customer rankings
`reports/products/`	Product performance
`reports/unpaid/`	Outstanding payments
`reports/inactive-customers/`	Churn risk
`reports/order-profitability/`	Per-order margin
`reports/export/*/`	CSV downloads
`reports/loyalty/*/`	Repeat, frequency, recency, LTV, cohorts

Nested URL pattern

DRF routers produce paths like:

- GET /api/orders/orders/ - list
- POST /api/orders/orders/ - create
- GET /api/orders/orders/{id}/ - detail
- PUT /api/orders/orders/{id}/ - update

The duplicate orders/orders comes from app prefix + router basename - consistent across apps.

9. Authentication & Authorization

Session-based auth flow

- POST /api/accounts/login/ with username/password (AllowAny)
- Django creates session; sets sessionid cookie

- Frontend stores user in React state (AuthContext); cookie handled by browser
- GET /api/accounts/me/ restores session on page refresh
- POST /api/accounts/logout/ destroys session

CSRF (Cross-Site Request Forgery)

- Django sets csrftoken cookie (readable by JS: CSRF_COOKIE_HTTPONLY = False)
- api.js interceptor copies cookie -> X-CSRFToken header on mutating requests
- Required for POST/PUT/PATCH/DELETE

CORS

- CORS_ALLOWED_ORIGINS includes http://localhost:3000
- CORS_ALLOW_CREDENTIALS = True - required for cookies cross-port

Permission classes (accounts/permissions.py)

```
| Class | Rule |
|-------|------|
| `IsAdmin` | `user.is_admin` |
| `IsOperator` | operator OR admin |
| `IsCook` | cook OR admin |
```

Frontend mirror (ProtectedRoute.js)

- requiredRole="admin" - strict admin
- requiredRole="operator" - admin or operator
- requiredRole="cook" - admin or cook

Defense in depth: Always enforce permissions on the backend; frontend guards are UX only.

10. React Frontend Deep Dive

10.1 React concepts mapped to this project

```
| React concept | Where you see it |
|---------------|------------------|
| **Component** | Each `pages/*.js` file |
| **Props** | Parent -> child data |
| **State** | `useState` in forms and lists |
| **Effect** | `useEffect` to load data on mount |
| **Context** | `AuthContext` - global user state |
| **Router** | `App.js` - URL -> component |
| **Conditional render** | Loading spinners, empty lists |
```

10.2 File roles

```
| File | Role |
|------|------|
| `index.js` | Mount `<App />` to DOM |
| `App.js` | All routes + `AuthProvider` wrapper |
| `contexts/AuthContext.js` | Login state, `useAuth()` hook |
| `services/api.js` | Single axios instance |
| `components/Layout.js` | Shell: header, sidebar, theme toggle |
| `components/ProtectedRoute.js` | Auth + role gate |
| `pages/*.js` | Screen logic (fetch, display, forms) |
| `styles/global.css` | Shared buttons, tables, forms |
| `components/Layout.css` | CSS variables, light/dark theme |
```

10.3 Route map

```
| Path | Page | Role |
|------|------|------|
| `/login` | Login | public |
| `/` | Dashboard | any authenticated |
| `/customers/*` | Customers CRUD | operator+ |
| `/catalog/*` | Products (not `/products`) | cook+ |
| `/offerings` | Daily offerings | operator+ |
| `/orders/*` | Orders | operator+ |
| `/payments` | Payment ledger | operator+ |
| `/reports` | Reports tabs | operator+ |
| `/analytics` | Loyalty analytics | operator+ |
| `/import` | Bulk import | admin |
| `/google-sync` | Google Sheets | admin |
| `/users` | User admin | admin |
```

Known quirk: Dashboard may link to /products/new; correct path is /catalog/new.

10.4 Typical page pattern

```
// 1. Import api and hooks
import api from '../services/api';
import { useState, useEffect } from 'react';

// 2. Load data on mount
useEffect(() => {
  api.get('/orders/orders/').then(res => setOrders(res.data));
}, []);

// 3. Render table + filters + actions
// 4. POST/PUT/DELETE on button click, then refresh or navigate
```

10.5 Styling approach

- CSS variables in Layout.css (--primary-color, etc.)
- Theme: data-theme="light"|"dark" on <html>, persisted in localStorage
- No CSS modules - one .css file per page
- INR formatting: toLocaleString('en-IN') in JS

11. Integrations & Data Import

11.1 Admin bulk import (two-step)

- Preview - POST /api/imports/preview/ (multipart file upload)
- Validates rows, returns errors + sample rows
- Confirm - POST /api/imports/confirm/ with mode=valid_only
- Transactional insert; writes ImportLog

Types: customers, products, orders, payments.

Templates: GET /api/imports/template/{type}/ and import_templates/*.csv.

11.2 Google Sheets sync

Setup:

- Google Cloud project + Sheets API enabled
- Service account JSON -> GOOGLE_CREDENTIALS_JSON env or google_credentials.json
- Share target sheet with service account email
- Configure GoogleSheetConfig with column letter mapping

Sync logic (summary):

- Read sheet from row 2+ (row 1 = headers)
- Skip rows already in GoogleSheetOrderRef
- Find/create customer by mobile
- Match product by name/size heuristics
- Create order (confirmed, payment pending) + line item
- Optional write-back of order number to sheet

11.3 Order CSV import (operator)

POST /api/orders/orders/import_csv/ - lighter path for exported sheet data.

12. Development Workflow

First-time setup

```
python -m venv SSCo
source SSCo/bin/activate          # Windows: SSCo\Scripts\activate
pip install -r requirements.txt
cd frontend && npm install && cd ..
python manage.py migrate
python seed_demo_data.py          # optional
```

Daily development

```
# Terminal 1
source SSCo/bin/activate
python manage.py runserver        # :8000

# Terminal 2
cd frontend && npm start            # :3000
```

Or: ./setup.sh (starts both).

Environment variables (.env)

```
| Variable | Purpose |
|----------|---------|
| DJANGO_SECRET_KEY | Cryptographic signing |
| DEBUG | True for development only |
| ALLOWED_HOSTS | Comma-separated hostnames |
| CORS_ALLOWED_ORIGINS | Frontend URL(s) |
| GOOGLE_CREDENTIALS_JSON | Inline service account JSON (optional) |
```

Frontend optional: REACT_APP_API_URL in frontend/.env if API not on localhost:8000.

Demo credentials

```
| User | Password | Role |
|------|----------|------|
| admin | admin123 | admin |
| operator | operator123 | operator |
| cook | cook123 | cook |
```

Making schema changes

```
# After editing models.py
```

```
python manage.py makemigrations
python manage.py migrate
```

Running tests

```
python manage.py test
# or
python manage.py test tests.test_core
```

13. Python -> Full-Stack Learning Path

This section is a 12-month study plan to grow from core Python to building apps like SoupsSnacks without relying on AI tools.

Phase 1 - Months 1-3: Django fundamentals

Goals: Build a CRUD API without React.

```
| Topic | Practice in this repo |
|-------|----------------------|
| Models & migrations | Add a field to `Customer`, migrate |
| Django shell | `python manage.py shell` - query `Order.objects.filter(...)` |
| Admin | Register a model in `admin.py`, explore filters |
| Class-based views | Read one `APIView` in `reports/views.py` |
| DRF serializers | Trace `OrderCreateUpdateSerializer` nested create |
| Permissions | Add a new `@action` with `IsAdmin` only |
```

Exercise: Add a Customer.tags CharField end-to-end: model -> migration -> serializer -> list in admin.

Resources: Official Django tutorial, DRF quickstart, William Vincent's books.

Phase 2 - Months 4-6: Frontend basics + API consumption

Goals: Understand how React talks to Django.

```
| Topic | Practice |
|-------|----------|
| HTTP methods | Use browser DevTools Network tab on create order |
| JSON shapes | Compare API response to serializer fields |
| React state | Add a filter to an existing list page |
| React Router | Add a simple static `/help` page |
| axios | Add one new GET call and display result |
```

Exercise: Add notes display to customer list without AI - read Customers.js and CustomerListSerializer.

Resources: React official docs (Learn section), MDN HTTP guide.

Phase 3 - Months 7-9: Full-stack features

Goals: Ship a small feature alone.

```
| Topic | Practice |
|-------|----------|
| Nested writes | Study order create payload in Network tab |
| Transactions | Read `imports/views.py` confirm flow |
| Aggregations | Add a simple count endpoint in `reports/` |
| File upload | Trace `Import.js` multipart form |
| Auth edge cases | Test logged-out redirect, wrong role |
```

Exercise: Add "favorite product" per customer (FK nullable) - backend + dropdown on order form + migration.

Phase 4 - Months 10-12: Production mindset

Goals: Deploy and maintain.

Topic	Learn
PostgreSQL	Switch DATABASES in settings for staging
Environment secrets	Never commit `.env` or `google_credentials.json`
Static files	`npm run build` + WhiteNoise or nginx
HTTPS	Required for secure cookies in production
Debugging	Django `LOGGING`, React error boundaries
Testing	Write one `APITestCase` for your new endpoint

Exercise: Deploy to a $5 VPS or Railway/Render with Postgres - even if only you use it.

Skills checklist (self-assessment)

- [ ] I can explain MVC vs Django's MTV pattern
- [ ] I can write a migration and fix a merge conflict
- [ ] I can add a DRF endpoint without copying boilerplate blindly
- [ ] I understand why CSRF exists and when cookies are sent
- [ ] I can read React DevTools and find which component fetched data
- [ ] I can trace a bug from UI click -> API -> serializer -> model
- [ ] I know when logic belongs in model vs serializer vs view
- [ ] I can deploy backend + frontend separately

Where logic should live (golden rules)

Layer	Put here
Model	Invariants, computed properties, save() side effects that always apply
Serializer	Input validation, nested create/update, API field shaping
View	HTTP concerns, permissions, filtering querysets
React page	Display, user input, calling API - not business rules

14. How to Extend This App

Add a new API resource (checklist)

- Create or extend model in appropriate app
- makemigrations + migrate
- Add Serializer(s) in serializers.py
- Add ViewSet in views.py with permission_classes
- Register router in app urls.py
- Include app URLs in soupssnacks/urls.py if new app
- Add axios calls in React page(s)
- Add route in App.js + nav link in Layout.js
- Set ProtectedRoute role
- Write at least one API test

Add a report

- New class in reports/views.py (subclass APIView)
- URL in reports/urls.py
- New tab or section in Reports.js or CustomerAnalytics.js

Add an import type

- Extend preview/confirm logic in imports/views.py
- Add template CSV in import_templates/
- Update Import.js type dropdown

15. Production & Security Checklist

Before going live:

- [ ] Set DEBUG=False
- [ ] Strong DJANGO_SECRET_KEY from env
- [ ] Restrict ALLOWED_HOSTS and CORS_ALLOWED_ORIGINS
- [ ] Use PostgreSQL (or MySQL), not SQLite
- [ ] HTTPS everywhere (cookies secure)
- [ ] Do not commit google_credentials.json or .env
- [ ] Run python manage.py check --deploy
- [ ] Serve React build/ via nginx or CDN; API on separate subdomain if needed
- [ ] Back up database regularly
- [ ] Pin all dependencies including optional ones in requirements.txt

16. Glossary

```
| Term | Meaning |
|------|---------|
| **SPA** | Single Page Application - React loads once, routes client-side |
| **DRF** | Django REST Framework |
| **ViewSet** | DRF class combining list/create/retrieve/update/destroy |
| **Serializer** | Converts models ? JSON; validates input |
| **Migration** | Versioned DB schema change |
| **CORS** | Browser security for cross-origin API calls |
| **CSRF** | Token preventing forged form submissions |
| **Session** | Server-side login state referenced by cookie |
| **FK** | ForeignKey - relational link between tables |
| **PROTECT** | on_delete - block delete if children exist |
| **CRA** | Create React App - webpack-based React starter |
```

17. Quick Reference Tables

Role -> feature matrix

```
| Feature | Admin | Operator | Cook |
|---------|:-----:|:--------:|:----:|
| User management | Yes | No | No |
| Imports / Google Sync | Yes | No | No |
| Customers, orders, payments, reports, offerings | Yes | Yes | No |
| Product read | Yes | Yes | Yes |
| Product write / costs | Yes | No | Yes |
```

Order status values

draft -> confirmed -> preparing -> ready -> delivered -> completed (or cancelled)

Payment status values

pending, partial, paid, refunded (auto-updated from Payment rows)

Key file index

Concern	Path
Settings	soupssnacks/settings.py
Root URLs	soupssnacks/urls.py
Permissions	accounts/permissions.py
API client	frontend/src/services/api.js
Routes	frontend/src/App.js
Google sync service	integrations/google_sheets_service.py
Import logic	imports/views.py
Reports	reports/views.py

Document maintenance

When you add major features, update this file and regenerate the PDF:

```
cd docs
python build_reference_pdf.py
```

End of reference document - SoupsSnacks v2